

Data and Architecture Standards

DAEN 459 / Spring 2025

Alexander Abuabara



Setting clear **standards** helps make the project easier to scale, repeat, and trust by following clear technical and organizational guidelines.

Defining a well-structured **system design and process standards** keeps the work consistent, protects data quality, and supports teamwork through every stage of the project.

Example "Performance Analytics Pipeline"

Without Standards

- ▶ Inconsistent file formats (CSV, JSON, Excel)
- ▶ No schema validation or version control
- ▶ Code varies across team members (hard to reproduce results)

With Clear Standards

- ▶ Defined **system design**: raw → processed data folders
- ▶ Standardized formats (Parquet), naming (`snake_case`)
- ▶ Schema validation and logging in every transformation
- ▶ Git workflow for version control and code reviews

Result: Reliable, scalable, and reproducible data pipeline with improved collaboration.

Data Governance

- ▶ **Ownership & Stewardship:** Assign clear data owners and stewards responsible for data quality, updates, and lifecycle management. Establish access permissions (read, write, admin) based on project roles.
- ▶ **Compliance:** Ensure all data handling aligns with relevant regulations such as **GDPR** (privacy), **FERPA** (student records), or **HIPAA** (health data). Document data sources, consent, and usage rights.
- ▶ **Version Control:** Track data and metadata changes using tools like **Data Version Control (DVC)** or **lakeFS**. Maintain reproducibility by linking data versions to specific code commits.
- ▶ **Data Retention:** Define policies for how long data is stored, archived, or deleted. Classify data by sensitivity and determine appropriate retention timelines to reduce storage costs and risk.
- ▶ **Auditability:** Keep detailed logs of data access, transformations, and modifications. Enable full traceability for accountability and compliance reporting.

Data Quality

Quality Dimensions:

- ▶ Accuracy
- ▶ Completeness
- ▶ Consistency
- ▶ Timeliness

Validation Techniques:

- ▶ Schema validation
- ▶ Automated profiling
- ▶ Data quality dashboards

Data Documentation & Metadata

- ▶ Maintain **metadata catalogues** (DataHub, Amundsen, OpenMetadata).
- ▶ Build **data dictionaries** defining fields, types, and logic.
- ▶ Capture **lineage tracking** (with [Apache Atlas](#) or [Marquez](#)).
 - Helps you answer questions like:
 - ▶ Where did this data originate?
 - ▶ What transformations were applied?
 - ▶ Which reports or models use this dataset?
- ▶ Document assumptions, transformations, and business rules (*what the data means and how it should behave according to the project's real-world context*).

Data Security & Privacy

- ▶ Encryption:
 - ▶ At rest: AES-256
 - ▶ In transit: TLS/SSL
- ▶ Mask or anonymize sensitive data (PII, PHI).
- ▶ Maintain access logs and audit trails.
- ▶ Implement role-based access control (RBAC).

Data Interoperability & Formats

- ▶ Prefer open, efficient formats: Parquet, Avro, ORC, JSON.
- ▶ Define **schema evolution policies** for backward compatibility.
Schema evolution policies define how your data structure (the schema: columns, types, fields) can change over time without breaking existing pipelines or applications.
- ▶ Maintain naming conventions and directory hierarchy:
 - ▶ `/raw`, `/working`, `/processed`

Architectural Design Principles

- ▶ **Scalability:** Horizontal scaling with distributed systems.
- ▶ **Modularity:** Independent ingestion, processing, storage, serving layers.
- ▶ **Fault Tolerance:** Retry logic, checkpointing, idempotent writes (i.e., writing the same data or running the same operation multiple times produces the same final result — no duplicates, no corruption, no unintended side effects).
- ▶ **Observability:** Logging, metrics, and tracing built-in.

Reference Architecture

- ▶ A **reference architecture** is a high-level blueprint or template that shows how different components of a system should be organized and interact with each other.
 - ▶ It's not an exact implementation, but rather a model or guide that helps teams design consistent, scalable, and maintainable systems.
1. **Ingestion Layer:** collects raw data from multiple sources (APIs, logs, databases).
 2. **Processing Layer:** cleans, transforms, and enriches the data.
 3. **Storage Layer:** stores data in organized zones
For instance: Raw → Curated → Analytics
 4. **Serving Layer:** delivers processed data to users or systems (dashboards, ML models, APIs, BI tools).
 5. **Governance Layer:** handles quality, security, lineage, and compliance across all stages.

Coding & Development Standards

- ▶ Follow [Python PEP8](#), SQL style guides, and naming conventions.
- ▶ Version control with Git: feature/develop/main branches.
- ▶ Code reviews through Pull Requests.
- ▶ Implement CI/CD for data workflows (GitHub Actions, Jenkins, Dagster).

Pipeline Design Patterns

- ▶ **ETL vs. ELT:** Select the right pattern based on project needs. ETL (Extract–Transform–Load) performs transformations before loading into storage — ideal for on-prem systems. ELT (Extract–Load–Transform) loads raw data first and transforms it within the data warehouse — better for cloud-scale analytics.
- ▶ **Idempotency:** Design every pipeline step so rerunning it produces the same final result. This prevents duplication, ensures reliability, and supports recovery from job failures.
- ▶ **Partitioning and Parallelization:** Split large datasets by logical keys (e.g., date, region, or customer ID) to process data in parallel, improving speed and scalability.
- ▶ **Job Orchestration:** Use workflow managers to schedule, monitor, and manage dependencies between tasks for consistent, automated data flow.

Performance Optimization

- ▶ **Define Performance KPIs:** Establish clear metrics such as query latency, data throughput, and cost efficiency to evaluate system performance and identify bottlenecks.
- ▶ **Caching and Indexing:** Implement caching for frequently accessed data and create indexes on high-usage columns to reduce read times and improve query speed.
- ▶ **Query Optimization:** Refactor queries to minimize full table scans, leverage partition pruning, and push computations closer to data sources for efficiency.
- ▶ **Resource Scaling:** Enable automatic scaling of compute resources to handle variable workloads, ensuring performance during peak demand without overspending.
- ▶ **Monitoring and Tuning:** Continuously monitor job performance and storage utilization using appropriate tools, and adjust configurations based on observed metrics.

Monitoring, Logging, & Alerting

- ▶ Maintain structured logs (INFO, WARN, ERROR).
- ▶ Define alert thresholds for performance and data quality issues.
- ▶ Include automated incident response procedures.

Documentation & Testing

- ▶ Architecture diagrams with draw.io or Lucidchart.
- ▶ Maintain README and internal wiki per pipeline.
- ▶ **Testing Framework:**
 - ▶ Unit tests for transforms
 - ▶ Integration tests for component flows
 - ▶ Regression tests for drift detection

Key Principles for the Project Charter

- ▶ **Consistency:** Standards create reproducible and auditable outcomes.
- ▶ **Quality:** Strong data governance ensures trust and usability.
- ▶ **Maintainability:** Documentation and modularity prevent technical debt.
- ▶ **Scalability:** Architecture designed for growth and efficiency.